

并行处理与体系结构(4)

Parallel Processing and Architecture

2025-4-20

1

回顾：并行机的分类

Michael Flynn 的分类

- ▶ SISD: Single Instruction and Single Data Stream (uniprocessor) 单指令操作单个数据元素
- ▶ SIMD: Single I and Multiple D Streams (GPUs) 单指令操作多个数据元素
- ▶ MISD: Multiple I and Single D Streams 多指令操作单个数据元素
- ▶ MIMD: Multiple I and Multiple D Streams (multicores) 多指令操作多个数据元素 (多指令流)

基于模型的分类

- ▶ 共享内存 (Shared-memory)
- ▶ 消息传递 (Message-passing)
- ▶ 数据流 (Dataflow)
- ▶ 脉动阵列 (systolic)
- ▶ 数据并行 (Data parallel)

▶

▶ 2

2

回顾：并行的类型

指令级并行

- ▶ 一个指令流中的不同指令可以并行执行
- ▶ 流水线, 乱序执行, 投机执行, VLIW
- ▶ 数据流

数据并行

- ▶ 数据的不同片段可以被并行的操作
- ▶ SIMD: 向量处理, 阵列处理
- ▶ 脉动阵列, 流处理器

任务级并行

- ▶ 不同的“任务/线程”可以被并行执行
- ▶ 多线程
- ▶ 多处理 (多核)

▶ 3

3

回顾：多处理器的类型

松耦合多处理器

- ▶ 没有共享的全局存储地址空间
- ▶ 多机网络
 - ▶ 基于网络的多处理器
- ▶ 通常通过消息传递编程
 - ▶ 通过显式调用 (send, receive) 通信

紧耦合多处理器

- ▶ 共享全局存储地址空间
- ▶ 传统的多处理: 对称多处理器 (SMP)
 - ▶ 现有的多核处理器, 多线程处理器
- ▶ 编程模型与单处理器类似 (多任务单处理器), 除了
 - ▶ 操作共享数据需要同步

▶ 4

4

回顾：紧耦合(共享内存)多处理器的主要难点

- ▶ 共享存储同步
 - ▶ 锁, 原子操作
- ▶ Cache 一致性
- ▶ 访存操作的序
 - ▶ 程序员希望硬件提供什么?
- ▶ 资源共享, 竞争和分区
- ▶ 通信: 互连网络
- ▶ 负载均衡

▶ 5

5

回顾：Amdahl定律的启示

▶ Amdahl定律的另一种表示

- ▶ f: 程序可并行化的比例
- ▶ N: 处理器数量

$$\text{加速比} = \frac{1}{1 - f + \frac{f}{N}}$$

- ▶ Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

- ▶ 最大化加速比受限于串行部分: 串行瓶颈
- ▶ 并行部分通常也不是完美的并行
 - ▶ 同步开销 (比如, 更新共享的数据)
 - ▶ 负载不均衡开销 (并行化不完美)
 - ▶ 资源共享开销 (N个处理器之间的竞争)

▶ 6

6

READING LIST

▶ 7 topics; 17 papers

▶ Topic I =====

Scalable Cache Coherence 第9周 (4月27日下午)

- 1a) Review: Chapter on Cache Coherence, "Parallel Computer Architecture A Hardware/Software Approach" by Culler and Singh, published by Morgan Kaufmann, 1997.
1b) D. Lenoski et al. "The Directory-Based Cache Coherence Protocol for the DASH Multiprocessor" International Symposium on Computer Architecture 1990.
1c) A. Gupta et al.
"Reducing Memory and Traffic Requirements for Scalable Directory-Based Cache Coherence Schemes", International Conference on Parallel Processing 1990.
1d) J. Torrellas, M. Lam & J. Hennessy, "False Sharing and Spatial Locality in Multiprocessor Caches", Transactions on Computers, June 1994.

▶ 7

7

ZY2457A03	范家诚
ZY2457A12	襄佳霖
ZY2457A14	盛一搏
ZY2457A16	张晓舟

READING LIST (2)

Topic 2 =====

Memory Consistency Models 第9周 (4月27日下午)

- 2a) K. Gharachorloo et al. "Memory Consistency and Event Ordering in Scalable Shared-Memory Multiprocessors", International Symposium on Computer Architecture 1990.
2b) K. Gharachorloo et al. "Two Techniques to Enhance the performance of Memory Consistency Models", International Conference on Parallel Processing 1991.
2c) S. Adve and K. Gharachorloo. "Shared Memory Consistency Models: A Tutorial", DEC WRL Research Report 95/7, 1995

▶ 8

8

ZY2457420	王鲁新
ZY2457422	李泽昊
ZY2457424	苗雨阳
ZY2457509	姬夏迎

READING LIST (3)

Topic 3 =====

Prefetching 第12周 (5月18日下午)

ZY2457318	戚永飞
ZY2457328	于浩喆
ZY2457329	张超骏
ZY2457415	刘鑫

- 3a) T. Mowry et al. "Design and Evaluation of a Compiler Algorithm for Prefetching", Architectural Support for Programming Languages and Operating Systems, 1992.
3b) Y. Solihin et al. "Using a User-Level Memory Thread for Correlation Prefetching", International Symposium on Computer Architecture 2002.

▶ 9

9

READING LIST (4)

Topic 4 =====

Synchronization 第12周 (5月18日下午)

ZY2457308	王康宁
ZY2457311	王振伟
ZY2457312	王志飞
ZY2457316	文砚雷

- 4a) J. Goodman et al. "Efficient Synchronization Primitives for Large Scale Cache-Coherent Multiprocessors", Architectural Support for Programming Languages and Operating Systems, 1989.
4b) J. Mellor-Crummey and M. Scott. "Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors", ACM Transactions on Computer Systems, 1991

▶ 10

10

READING LIST (5)

Topic 5 =====

Multithreading 第13周 (5月25日下午)

ZY2457213	李家兵
ZY2457216	李尚原
ZY2457229	邱文凯
ZY2457233	任星舟

- 5a) D. Tullsen et al. "Simultaneous multithreading: Maximizing On-Chip Parallelism", International Symposium on Computer Architecture 1995.
5b) D. Tullsen et al. "Exploiting Choice: Instruction Fetch and Issue on an Implementable Simultaneous Multithreading Processor", International Symposium on Computer Architecture 1996.

▶ 11

11

READING LIST (6)

Topic 6 =====

Multiple Processors on a Chip 第13周 (5月25日下午)

ZY2457118	程晓
ZY2457124	段泽邦
ZY2457129	郭燕
ZY2457206	沈紫静

- 6a) K. Olukotun et al. "The Case for a Single-Chip Multiprocessor", Architectural Support for Programming Languages and Operating Systems, 1996
6b) G. Sohi et al. "Multiscalar Processors", International Symposium on Computer Architecture 1995.
6c) V. Krishnan and J. Torrellas. "A Chip Multiprocessor Architecture with Speculative Multithreading", IEEE Transactions on Computers 1999

▶ 12

12

READING LIST (7)

Topic 7
=====

Speculative Parallelization and Execution 第13周 (5月25日下午)

- 7a) J. Steffan et al. "A Scalable Approach to Thread-Level Speculation" International Symposium on Computer Architecture 2000.
7b) J. Martinez et al. "Speculative Synchronization: Applying Thread-Level Speculation to Explicitly Parallel Applications", Architectural Support for Programming Languages and Operating Systems, 2002.

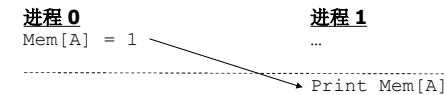
ZF2406102	盖一鸣
ZF2406108	罗晖
ZF2406114	张峻玮
ZY2457103	段欣然
ZY2457104	姜明月

▶ 13

13

共享存储模型

- ▶ 很多并行程序通过共享存储通信
- ▶ 进程0写某个地址，接着进程1读
- ▶ 两个进程之间通信



- ▶ 每一个读操作都能够收到任何人最后一次写的值
- ▶ 这需要同步 (最后一次写是什么意思?)
- ▶ 如果Mem[A]在cache中(两个进程端都有)会怎么样?

▶ 14

14

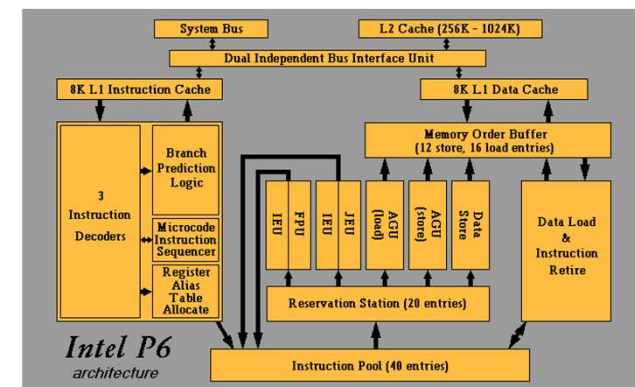
一致性

- ▶ 很多现代计算机系统和绝大多数多核芯片都在硬件层面支持共享存储
- ▶ 在共享存储系统中，每个处理器核都可以读或写同一个共享的地址空间
- ▶ 对于共享存储的机器，存储一致性模型定义了其存储系统的“可见”行为
- ▶ 一致性模型给出了“存取”规则以及它们对存储器的行为
- ▶ 作为存储一致性模型的一部分，大多数机器也提供cache一致性协议用来保证多个缓存的拷贝是最新的

▶ 15

15

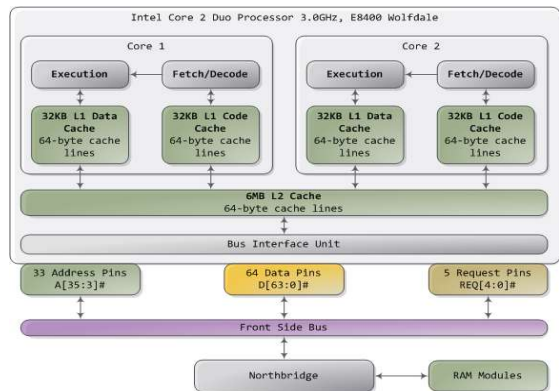
CPU Architecture(复杂版)



▶ 16

16

CPU 体系结构(简化版)



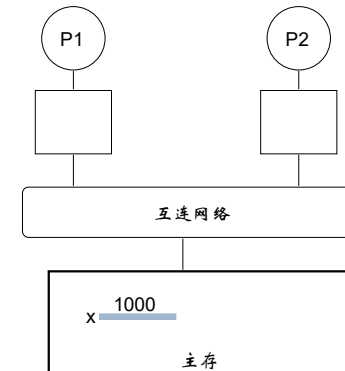
<http://duartes.org/gustavo/blog>

► 17

17

Cache 一致性

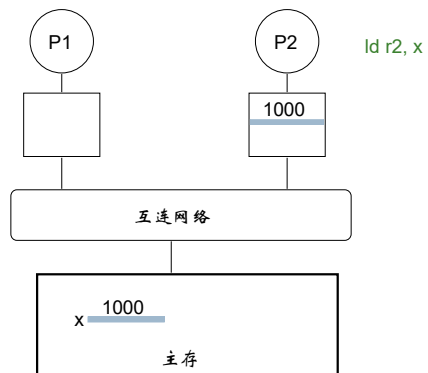
- 基本问题: 如果多处理器cache同一个块, 如何保证它们看到的是一致的状态?



► 18

18

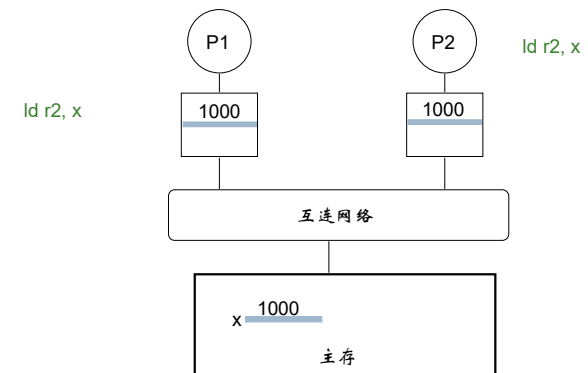
Cache一致性问题——例子



► 19

19

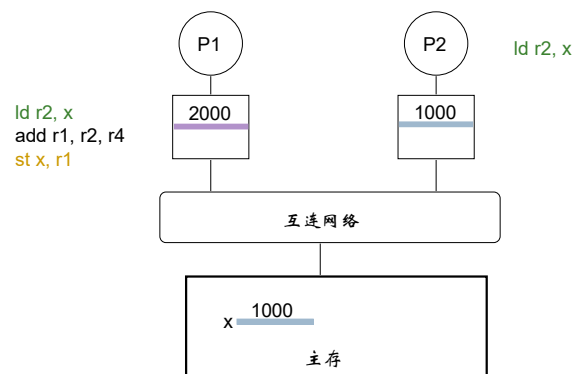
Cache一致性问题——例子



► 20

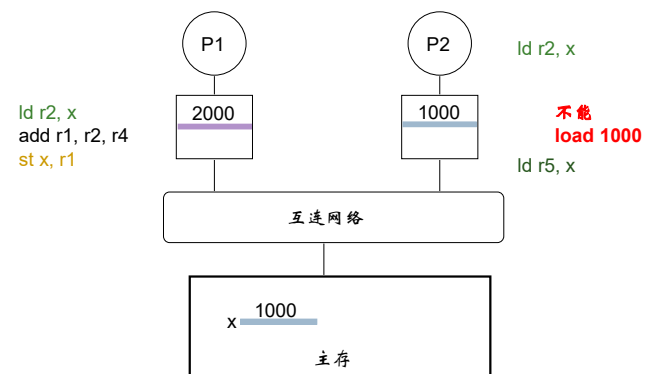
20

Cache一致性问题——例子



21

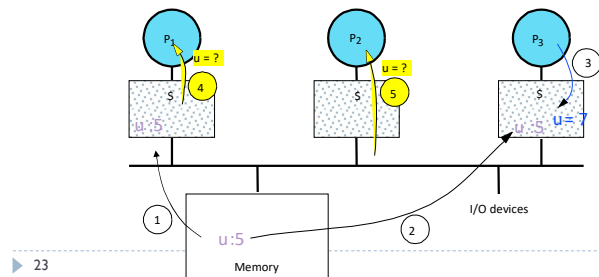
Cache一致性问题——例子



22

Cache 不一致的原因

- Cache对于现代高速处理器是至关重要的
- 一个cache block的多个拷贝很容易引发不一致
 - 共享可写数据
 - 进程迁移
 - I/O操作（绕过Cache的I/O操作）



23

Cache 一致性: 该由谁负责?

- 基于软件vs.基于硬件
- 软件
 - Cache对软件不可见，程序员还能够保证一致性吗?
 - 可以有或者没有硬件协助
- 硬件
 - 简化软件的工作

24

Cache 一致性解决方案

▶ 完全不依靠硬件的一致性

- ▶ 保持cache一致性是软件的责任
 - ▶ 基于编译器或运行时系统支持

+ 让微架构更简单

-- 使程序员的工作更困难

- ▶ 需要考虑硬件cache以保持程序的正确性?

-- 软件中保证一致性会带来额外开销

- ▶ 很困难的问题, 需要完整的信息
 - 访存行为的混乱、显式的并行

▶ 所有的处理器共享所有的cache

+ 不需要一致性

-- 共享的cache成为瓶颈

-- 这种方案下极难设计即具备低延迟cache又有可扩展性的系统

▶ 更多关注于基于硬件的方案

▶ 25

25

保持一致性

▶ 需要保证所有处理器看到相同存储位置的值的一致性(一致更新)

▶ P0向位置A的写入应该被P1 (最终)看到, 所有对A的写应该按照某种序呈现出来

▶ 一致性需要提供:

▶ 写的传播: 保证更新被传播出去

▶ 写的序列化: 为所有处理器提供一致的全局序

▶ 需要一个全局的序列化点对写排序

▶ 26

26

硬件解决方案

▶ 基本思路:

▶ 一个处理器/cache向所有其它处理器广播它对某个内存位置的写/更新

▶ 另一个拥有这个位置的数据的cache要么更新要么置无效它的本地拷贝

▶ 分类

- ▶ 基于监听vs.基于目录的模式
- ▶ 写直达vs.写回(基于所有权) 协议
- ▶ 更新vs.置无效 协议
- ▶ 脏共享vs.无脏共享协议

▶ 27

27

写直达模式

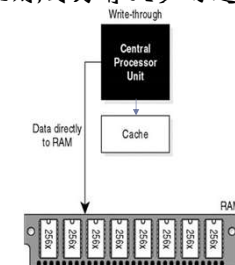
▶ 所有的处理器写会导致

▶ 本地cache的更新以及全局总线写, 以实现

▶ 更新主存储器

▶ 置无效/更新所有其它cache中该块的拷贝

▶ 基本没有人采用, 因为太多的通信

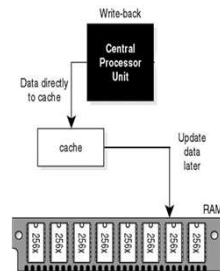


▶ 28

28

写回/所有权模式

- ▶ 当某个处理器拥有一个块的所有权时，处理器写不会导致总线写，能够有效节省带宽
- ▶ 今天大多数基于总线的多处理器使用这种模式
- ▶ 有很多基于所有权的协议变种存在



▶ 29

29

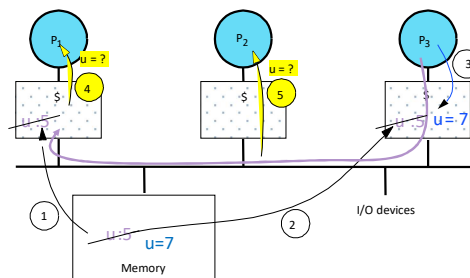
置无效vs.更新

- ▶ 置无效：写一个块时，所有其它cache中该块的拷贝置为无效
- ▶ 更新：写一个块时，所有其它cache中该块的拷贝都更新
- ▶ 置无效策略更为常用

▶ 30

30

例子：写直达置无效



▶ 31

31

一致性：更新vs. 置为无效

- ▶ 如何安全的更新有多份副本的数据？
 - ▶ 选择 1 (更新)：向所有拷贝推送一个更新
 - ▶ 选择 2 (置无效)：确保只有一个有效拷贝 (本地的)，更新它
- ▶ 读数据时：
 - ▶ 如果本地拷贝无效，发出请求
 - ▶ (如果另一个节点有有效拷贝，该节点返回，否则由内存返回)

▶ 32

32

一致性: 更新vs. 置为无效

▶ 写数据时:

- ▶ 按照之前的方法读一个块到cache

更新协议:

- ▶ 写一个块, 同时广播写的的数据给该数据的共享者
- ▶ (其它节点上如果有该数据在cache中则更新cache)

置无效协议:

- ▶ 写一个块, 同时广播需要置为无效的地址给数据共享者
- ▶ (其它节点对cache块作相应处理)

▶ 33

33

更新 vs. 置无效的Tradeoffs

▶ 目标是什么?

- ▶ 写的频率和共享行为都是至关重要的

▶ 更新

- + 如果共享者集合是常数并且并不经常有更新操作, 可以避免被置无效数据重新获取的开销 (广播更新模式)
- 如果其它核重写的的数据并没有被读取, 更新就是无用的
- 写直达cache策略 → 总线成为瓶颈

▶ 置无效

- + 置无效广播后, 核对数据有独占访问权
- + 只有在每个写之后会持续读的核会保有一份拷贝
- 如果写竞争度很高, 会导致ping-pong 效应(快速的互相置无效/重取)

▶ 34

34

两种cache一致性方法

- ▶ 如何确保合适的cache被更新?

▶ 监听总线(Snoopy Bus) [Goodman ISCA 1983, Papamarcos+ ISCA 1984]

- ▶ 基于总线, 所有请求在单点序列化
- ▶ 处理器观察其它处理器的动作
 - 比如: P1 在总线上发出对A的“排他读”请求, P0 看到后将自己的A的拷贝置为无效

▶ 目录(Directory) [Censier & Feautrier, IEEE ToC 1978]

- ▶ 每个块单点序列化, 序列化点分布在各节点
- ▶ 处理器生成对块的显式请求
- ▶ 目录追踪每个块的所有权(共享者集合)
- ▶ 目录协调置无效操作
 - 比如: P1 向目录请求排他的拷贝, 目录要求 P0 置无效相关数据, 等待应答, 然后向 P1 响应请求

▶ 35

35

基于监听的cache一致性

▶ 基于监听的分布式cache一致性模型

- ▶ 在全局总线上观察所有的行为
- ▶ 或者由某些全局广播机制告知相关的行为
- ▶ 是商用多处理器中更多采用的一种模式

▶ 36

36

监听Cache一致性

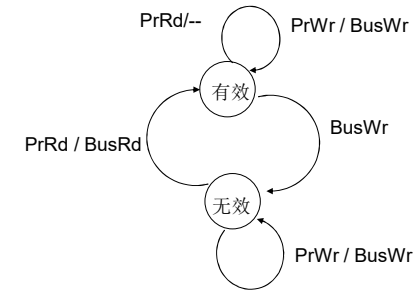
- 思路:
 - 所有cache“监听”所有其它cache的读/写请求,并保持cache块的一致性
 - 每个cache块在每个cache的标签存储中关联“一致性元数据”
- 当所有cache共享一个公共总线时很容易实现
 - 每个cache在总线上广播它的读/写操作
 - 对于小规模的多处理器很有效
 - 如果是1000个节点的多处理器会怎么样?

▶ 37

37

一个简单的监听cache一致性协议

- Cache“监听”(观察)彼此的写/读操作,如果一个处理器写一个块,所有其它的处理器将自己cache中的相同块置为无效
- 一个简单的协议:



▶ 38

38

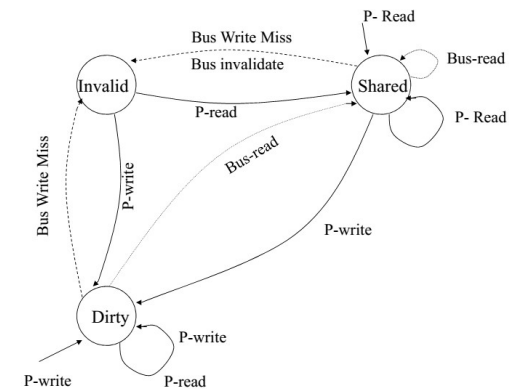
更复杂的协议: MSI

- 将每个块的单个有效位扩展为3个状态:
 - M**(odified): cache行是唯一拷贝并且被修改过(dirty)
 - S**(hared): cache行是一系列拷贝中的一个
 - I**(nvalid): 无效
- 读缺失导致总线上一个读请求,状态迁移到**S**
- 写缺失导致一个排他读请求,状态迁移到**M**
- 当一个处理器监听到从另一个处理器发出的排他读请求,它必须置无效自己的拷贝(如果有的话)

▶ 39

39

MSI



▶ 40

40

MSI协议的问题

- ▶ 一开始任意一个块都不在cache中
- ▶ 问题: 当读一个块时, 这个块立即变为S状态, 即使它可能只是cache中仅有的一个拷贝
- ▶ 为什么这是一个问题?
 - ▶ 设想一下, 读这个块的cache要在某个时刻写它
 - ▶ 即使它拥有的是唯一拷贝, 也需要向总线广播“置无效”!
 - ▶ 如果cache知道系统中只有它有这份拷贝, 它在写块的时候无需通知其它cache → 省去不必要的广播

▶ 41

41

解决方案: MESI

- ▶ 思路: 增加一个状态表示cache中仅有的拷贝, 并且是干净的
- ▶ 独占(Exclusive) 状态
- ▶ 如果总线读(BusRd)时, 没有其它cache有拷贝, 块就会被置为独占(E)状态
- ▶ 写块时可能会导致状态从独占(E) → 修改(M) 的变换!
- ▶ MESI 也称为 Illinois 协议(模式) [Papamarcos and Patel, ISCA 1984]

▶ 42

42

Illinois模式(MESI)

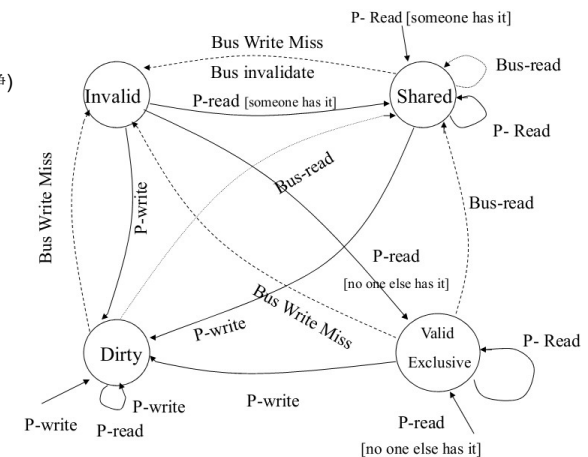
- ▶ 状态: I, VE(有效-独占), VS(有效-共享), D(脏)
- ▶ Cache知道自己是否拥有一个VE状态的块
- ▶ 在VE状态下写命中时不会发生置无效的通信
- ▶ 优点
 - ▶ 非常接近在单处理器上处理串程序的通信量
- ▶ 缺点
 - ▶ 更复杂

▶ 43

43

MESI(Illinois 模式)

四种状态:
M(独占拷贝, 脏)
E(独占拷贝, 干净)
S(共享, 干净)
I(无效)



▶ 44

44

MESI的问题

- ▶ 共享(S)状态需要数据是干净的
 - ▶ 即,所有拥有这个块的cache必须都拥有最新的拷贝,并且跟内存中的一致
- ▶ 问题:当有总线读时,如果块处于修改(M)状态需要将块写回内存
- ▶ 为什么这是个问题?
 - ▶ 内存可能做不必要的更新 → 其它处理器可能会在cache之后写块

▶ 45

45

改进MESI

- ▶ 思路 1: 总线读时不做M→S的状态转换,将拷贝置为无效,直接把修改过的块发给请求的处理器而不更新内存
- ▶ 思路 2: 做M→S的转换,但是指定一个cache作为拥有者(O),它负责在块被移除的时候写回
 - ▶ 这时候“共享(S)”意味着“共享的并且可能是脏的”
 - ▶ 这是MOESI 协议的一个版本

▶ 46

46

复杂Cache一致性协议中的tradeoff

- 协议可以通过更多的状态和预测机制优化
 - + 减少不必要的置无效和块的传输
- 当然,更多的状态和优化
 - 设计和验证更困难(导致更多需要考虑的情况和条件)
 - 收益递减

▶ 47

47

基于目录的cache一致性

- ▶ 核心思想: 在内存中维护一个全局目录,保存每个处理缓存了哪些位置的数据以及它们的状态
- ▶ 动机: 监听模式的可扩展性问题,因为它依赖于广播
- ▶ 目录模式更适合扩展
 - ▶ 通过跟踪缓存了同一内存空间数据的节点避免广播,使用点到点的通行保持一致性
 - ▶ 可以灵活选择任何可扩展的点到点网络

▶ 48

48

基于目录的cache一致性

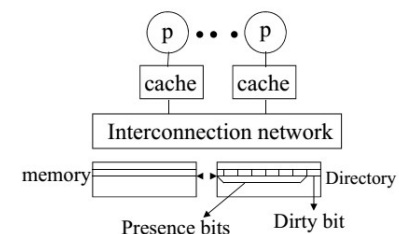
- ▶ 思路: 维护一个逻辑上的中心目录并保持跟踪每个cache块所在的位置, Cache查询这个目录以确保一致性
- ▶ 例子:
 - ▶ 对内存中的每个cache块, 在目录中存储 $P+I$ 位
 - ▶ 每个处理器使用 I 位, 显示这个块是否在那个处理器cache中
 - ▶ 独占位: 显示某个cache拥有这个块的唯一拷贝, 可以不通知其它cache而更新该块
 - ▶ 当读一个块时: 置目录中该块的对应cache位并向相应cache提供数据
 - ▶ 当写一个块时: 对所有拥有该块的其它cache执行置无效操作, 并重置它们在目录中的相应位
 - ▶ 为每一个cache中的每个块关联一个独占位

▶ 49

49

基本模式(Censier & Feautrier)

- ▶ 假设有 k 个处理器
- ▶ 对内存中的每个cache块, 有 k 个presence位和 I 个dirty位
 - ▶ 读miss
 - ▶ 写miss
 - ▶ 对有效(无拥有者)数据的写命中



▶ 50

50

扩展目录的一些问题

- ▶ 目录该有多大?
- ▶ 如何减小目录的访问延迟?
- ▶ 如何将系统扩展到上千个节点?
- ▶ 内存和目录带宽的扩展
 - ▶ 不适合采用集中式的目录存储
 - ▶ 需要分布式cache一致性协议
- ▶ 目录存储的扩展性不好
 - ▶ 处理器节点增加时需要增加presence位的数量
 - ▶ 同时, 内存越大, 目录存储也越大

▶ 51

51

目录的结构

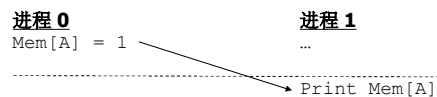
- ▶ 基于内存的模式(DASH)vs. 基于cache的模式(SCI)
- ▶ 基于cache的模式(或基于链表的)
 - ▶ 单链表
 - ▶ 双链表(SCI)
- ▶ 基于内存的模式(或基于指针的)
 - ▶ 全映射(Dir-N)vs. 局部映射(Dir-i-B, Dir-i-CV-r, ...)
 - ▶ 密集(DASH)vs. 稀疏目录模式

▶ 52

52

回顾：共享存储模型

- ▶ 很多并行程序通过共享存储通信
- ▶ 进程 0 写某个地址，接着进程 1 读
- ▶ 两个进程之间通信

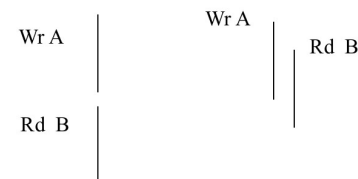


- ▶ 每一个读操作都能够收到任何人最后一次写的值
- ▶ 这需要同步 (最后一次写是什么意思?)
- ▶ 如果 Mem[A] 在 cache 中 (两个进程端都有) 会怎样?

53

Memory Consistency

- ▶ 隐藏访存延迟
- ▶ 多处理器结构下可能引发正确性问题
 - ▶ 访存的顺序约束
 - ▶ 读取的数值能够正确返回



54

单处理器存储模型

- ▶ 访存的原子性和程序序
- ▶ 无需考虑顺序就能保证正确性
 - ▶ 硬件: 缓冲, 流水线
 - ▶ 软件: 寄存器分配, 代码移动
- ▶ 对程序员简单
- ▶ 可以获得较高性能

55

共享存储多处理器

- ▶ 对于不同位置的访问顺序至关重要
- ▶ 不安全的重排序
 - ▶ 按照顺序发出的访问可能被观察到是乱序执行的
- ▶ 同一个位置的多个拷贝

56

操作的序

- ▶ 操作:A, B, C, D
 - ▶ 硬件该按照什么顺序执行(报告结果)这些操作?
- ▶ 程序员和微架构之间的约定
 - ▶ 由ISA确定
- ▶ 维护一个“期望的”序能够使程序员的人生更加美好
 - ▶ 调试; 状态恢复和处理异常
- ▶ 维护一个“期望的”序通常使硬件设计者的人生更加苦逼
 - ▶ 尤其是需要设计高性能处理器: 乱序执行中的Load-store队列

57

单处理器中访存的序

- ▶ 由冯诺依曼模型指定
- ▶ 顺序序
 - ▶ 硬件 **执行** load和store操作, **按照程序串行执行所指定的序**
- ▶ 乱序执行不改变语义
 - ▶ 硬件 **交付 (向软件报告结果)** load和store操作, **按照程序串行执行所指定的序**
- ▶ 优点: 1) 执行过程中的体系结构状态是精确的 2) 程序每次运行的体系结构状态是一致的 → 容易调试程序
- ▶ 缺点: 保持序会增加开销, 降低性能

58

数据流处理器中访存的序

- ▶ 当操作数准备好就可以执行访存操作
- ▶ 序由数据相关性指定
- ▶ 如果两个操作没有相关性, 他们可以按照任何序执行和交付
- ▶ 优点: 很多的并行性 → 高性能
- ▶ 缺点: 同一个程序每次执行可能会出现不同的序 → 很难调试

59

MIMD 处理器中访存的序

- ▶ 每个处理器的访存操作按照在该处理器上运行的“线程”的顺序执行序(假设每个处理器遵循冯诺依曼模型)
- ▶ 多处理器并发执行访存操作
- ▶ 存储器看到来自所有处理器的访问的序是什么样的?
 - ▶ 换句话说, 跨不同处理器的操作的序是什么样的?

60

为什么这件事很重要?

- ▶ 容易调试
 - ▶ 同一个程序在不同时间的执行拥有同样的执行序是很有用的
- ▶ 正确性
 - ▶ 如果从不同处理器看到的访存操作序不同, 是否会导致错误的操作呢?
- ▶ 性能和开销
 - ▶ 在实现提升性能相关技术(比如乱序执行、cache等)的同时执行严格的“顺序执行序”, 对硬件设计者是一个挑战

61

61

保护共享数据

- ▶ 线程不能够并发地更新共享数据
 - ▶ 为了正确性
- ▶ 对共享数据的访问被封装在临界区 或者通过 同步机制 (锁, 信号量, 条件变量)
- ▶ 只能有一个线程在某个确定的时间执行临界区
 - ▶ 同步互斥原则
- ▶ 多处理器需要提供同步原语的正确执行以确保程序员能够保护共享数据

62

62

支持同步互斥

- ▶ 程序员需要确认同步互斥是否正确的实现
 - ▶ 我们假设是这样的
 - ▶ 但是, 并行编程的正确性是一个重要的主题
 - ▶ 阅读: Dijkstra, “Cooperating Sequential Processes,” 1965.
 - ▶ Dekker的同步互斥算法
- ▶ 程序员依赖硬件原语支持正确的同步
 - ▶ 如果硬件原语不正确(或不确定), 程序员的人生。。。
 - ▶ 如果硬件原语是正确的, 但是不容易使用, 程序员的人生还是。。。

63

63

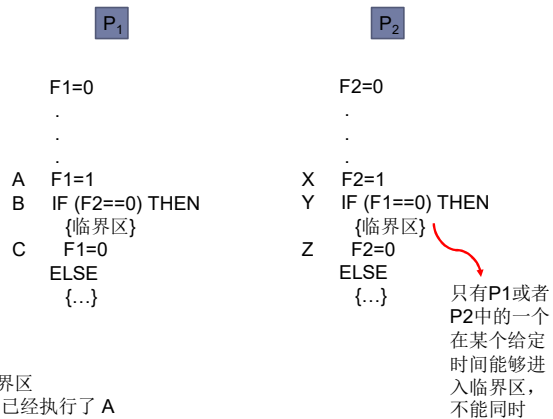
严格的一致性

- ▶ 如果存储系统中的Load总能返回相同地址最近一个Store的值, 就满足严格一致性
 - ▶ 只有一个副本
 - ▶ 访存在时间上没有重叠, 有全局时间序
 - ▶ Store原子性
 - ▶ 同步机制
- ▶ Store原子性
 - ▶ 如果不同线程永远无法同时观察到同一内存空间的不同值, 那么这一Store操作具备原子性
- ▶ Store原子性的充分条件
 - ▶ 对所有地址的Store操作可以确保形成一个全局序
 - ▶ 所有Load操作都必须全局完成
- ▶ 访存完成
 - ▶ 当线程i的Load无法获取到Store操作之前的值时, 该Store操作对于线程i完成
 - ▶ 当Store操作对于所有线程完成时, Store操作全局完成
 - ▶ 当Load值已经锁定且不能取消时, Load操作完成
 - ▶ 当Load操作完成, 且产生该Load值的Store操作已经全局完成时, 该Load操作全局完成

64

64

保护共享数据



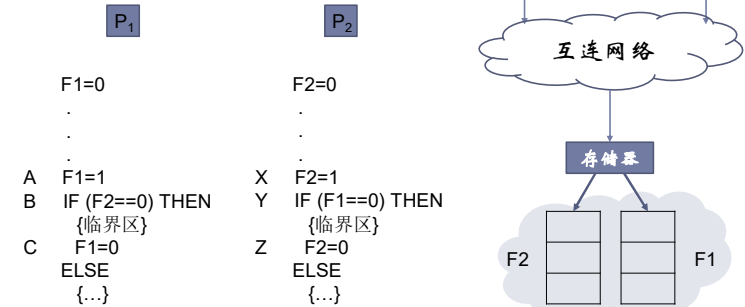
假设 P1在临界区
也就是说, P1已经执行了 A
这意味着F1必须是1
这意味着P2不能进入临界区

65

65

问题

- ▶ 两个处理器(遵循冯诺依曼模型)有可能在相同的某个时间进入临界区吗?
- ▶ 答案: 是的



▶ 两个处理器看到的是访存操作不同的序

66

66

该如何解决这个问题?

- ▶ 思路: 顺序一致性(Sequential consistency)
- ▶ 所有处理器看到相同的访存操作的序
- ▶ 即, 所有的访存操作按照一个对所有处理器都一致的序执行 (称为全局总序)
- ▶ 假设: 在全局序中, 每个处理器自己的操作按顺序序执行

67

67

顺序一致性

- Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," IEEE Transactions on Computers, 1979
- ▶ 一个多处理器系统是顺序一致的, 如果:
 - ▶ 所有操作的结果都是相同的, 就好像所有处理器的操作都按照某种顺序序执行
 - 并且
 - ▶ 每个处理器操作所呈现出的序是按照程序所指定的序
- ▶ 这是一个访存执行序模型, 或者说是一个内存模型
 - ▶ 由ISA指明

68

68

顺序一致性操作序

▶ 可能正确的全局序(都是正确的):

- ▶ A B X Y
- ▶ A X B Y
- ▶ A X Y B
- ▶ X A B Y
- ▶ X A Y B
- ▶ X Y A B

▶ 什么序(交叉存取)会被观察到取决于具体实现和动态的延迟

69

69

如何保证

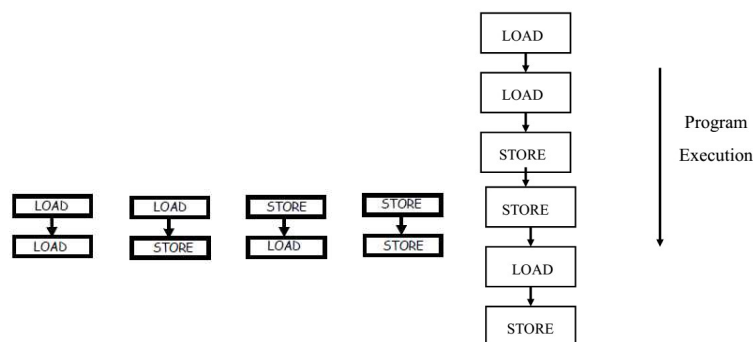
- ▶ 跟踪处理器发出的指令, 保持指令间序的约束
 - ▶ 当观察到读/写操作时
 - ▶ 处理器要跟踪自己发出的请求的执行情况
- ▶ 对某个内存位置的写操作设置约束
 - ▶ 原子的
- ▶ 定义存储模型的充分条件
- ▶ 一个处理器对同一个变量的访问不能乱序

70

70

顺序一致性

- ▶ 一个load执行之前, 所有之前的load/store必须执行完
- ▶ 一个store执行前, 所有之前的load/store必须执行完



71

71

顺序一致性的结果

▶ 推论

1. 在同样的执行中, 所有处理器会看到同样的访存操作全局序
→ 没有正确性问题
2. 对于不同的执行, 可能观察到不同的全局序 (每一个都是顺序一致的)
→ 调试仍然困难 (不同的执行序会变化)

72

72

顺序一致性的问题?

- ▶ 性能提升技术会使顺序一致性实现变得困难
- ▶ 乱序执行
 - ▶ Load之间乱序, load和独立的store之间乱序
- ▶ 高速缓存
 - ▶ 一个内存位置现在会出现在多个地方
 - ▶ 妨碍store的结果被其它处理器看到

73

73

顺序一致性的问题?

- ▶ 写缓冲的问题
 - ▶ 顺序一致性无法从写缓冲中受益
 - ▶ 编译器的问题
 - ▶ 再来看点到点同步
- ```
P1 P2
... ...
A:=1; while(flag==0)do nothing;
flag:=1; print A;
... ...
```
- ▶ 因为在 flag 和 A 之间没有相关性,编译器可能在 P1 上对两条指令重排序
  - ▶ 也可能在 P2 上 消除关于 flag 的循环
- 下一个思路: 改变内存一致性模型的规则可能依赖也可能不依赖同步

74

## 顺序一致性的改进

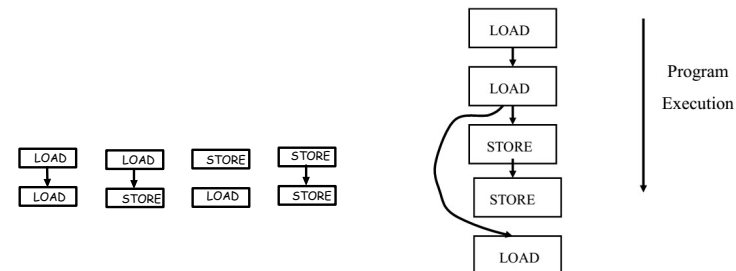
- ▶ 很漂亮的编程抽象,但是有两个问题:
  - ▶ 对序的要求过于保守
  - ▶ 限制了那些性能改善技术的激进性
- ▶ 全局序的要求是否太强了?
  - ▶ 是否需要一个对所有处理器的所有操作的全局序?
  - ▶ 仅针对所有store的全局序怎么样?
    - ▶ 完全store序访问模型;唯一-store序访问模型
  - ▶ 仅仅在同步的边界处执行全局序怎么样?
    - ▶ 松弛访问模型
    - ▶ 请求-释放一致性模型

75

75

## 处理器一致性

- ▶ 核心思想: load可以旁路store
- ▶ 一个load执行前, 所有之前的load必须执行完
- ▶ 一个store执行前, 所有之前的load/store必须执行完



76

76

## 弱的内存一致性

- ▶ 操作的序十分重要，尤其是当序会影响对共享数据的操作时 → 即，当处理器需要同步对某个“程序区域”的执行时
- ▶ 弱一致性
  - ▶ 思路：程序员指明在访存操作中不需要按序的程序区域
  - ▶ “内存栅障”指令描述了这些区域
    - ▶ 所有在栅障之前的内存操作必须在栅障被执行前完成
    - ▶ 所有在栅障之后的内存操作必须等待栅障执行完毕才能执行
    - ▶ 栅障按程序序执行
  - ▶ 所有同步操作的表现就像栅障

77

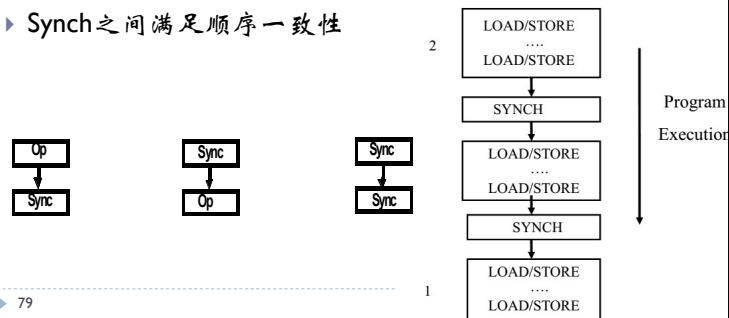
## 弱一致性

- ▶ 假设我们在临界区内
- ▶ 可以采用流水执行，因为程序员可以保证在离开临界区之前没有其它进程可以使用相关的数据结构
- ▶ 优点：更高的性能（更多的overlap）
- ▶ 缺点：需要区分普通的load/store和同步Synch

78

## 弱一致性

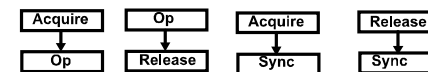
- ▶ 一个普通的load/store执行前，所有之前的synch必须执行完
- ▶ 一个synch执行前，所有之前的普通load/store必须执行完
- ▶ Synch之间满足顺序一致性



79

## 释放一致性

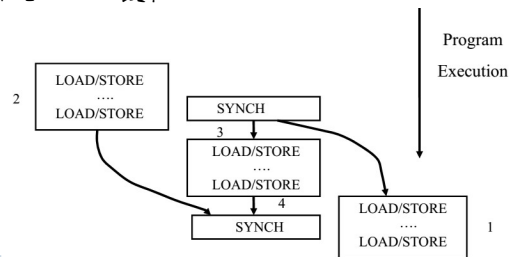
- ▶ 区别
  - ▶ Synch获取：lock
  - ▶ Synch释放：unlock
- ▶ 跟在释放之后的Load/store无需等到释放完成
- ▶ 获取无需等待之前的load/store完成
- ▶ 临界区内的访问无需等待临界区外的load/store



80

## 释放一致性

- ▶ 普通的load/store执行之前，所有之前的synch获取必须完成
- ▶ Synch释放执行之前，所有之前的普通load/store必须完成
- ▶ 获取/释放按照处理器一致性



▶ 81

81

## Tradeoff: 更弱的一致性

- ▶ 优点
  - ▶ 不需要保证访存操作非常严格的序
    - 使得一些性能提升技术的硬件实现更简单
    - 可以比严格的序性能更高
- ▶ 缺点
  - ▶ 程序员(或者软件)的负担更重(需要保证“栅栏”正确)
- ▶ 程序员-微架构折衷的又一个例子

▶

82

82

谢谢!

▶ 83

83